

Ćw. 11b. Budowa modelu generującego tekst. Architektura DekoderOnlyTransformer

Transformer to architektura sieci neuronowej, która została zaproponowana w artykule „Attention is All You Need” autorstwa Ashisha Vaswaniego i in. w 2017 r. Jest to bardzo pomysłowa architektura, która doprowadziła do wielu najnowocześniejszych wyników w ciągu ostatnich kilku lat. Może być używana do generowania tekstu (GPT-3), tworzenia pięknych obrazów z tekstu (Imagen, Dall-e 2, Stable diffusion), komponowania muzyki (music transformer), zamiany mowy na tekst (Whisper), rozumienia struktury białka (AlphaFold), uczenia samochodów, aby same prowadziły (Tesla FSD), a nawet uczenia się wykonywania wielu różnych zadań (Gato).

W tym kontekście duże modele językowe (LLM) będą miały wielki wpływ w przyszłości. Prawdopodobnie będziemy dużo wchodzić w interakcje z tymi modelami w przyszłości, od burzy mózgów mającej nam pomóc w kreatywności, pomagającej zrozumieć duże ilości informacji. GPT-3 już pomaga w kodowaniu i pisaniu (również na potrzeby tego wpisu na blogu), a te modele będą się naprawdę szybko poprawiać. Dlatego dobrze jest zrozumieć, jak działają.

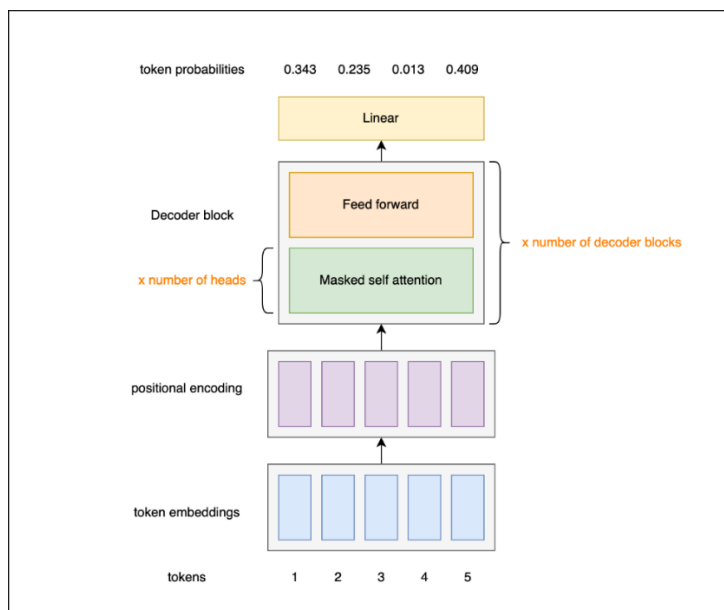
W tym wpisie na blogu zostanie pokazane, jak zbudować model generowania tekstu od podstaw, korzystając z architektury transformera. Celem tego wpisu nie jest zbudowanie najlepszego modelu generowania tekstu, ale próba uczynienia każdego kroku jego budowy tak przejrzystym, jak to możliwe.

Wstęp

Modele generowania tekstu, takie jak GPT-3, są budowane przy użyciu architektury transformera.

W przeciwieństwie do typowych modeli sequence-to-sequence używanych do zadań translacji, które mają zarówno koder, jak i dekodery, modele generowania tekstu wymagają tylko dekodera. Dzieje się tak, ponieważ sekwencje wejściowe i wyjściowe są zasadniczo takie same — model generuje tekst token po tokenie na podstawie poprzedniego kontekstu.

Architektura tylko dekodera wygląda następująco:



W przypadku generowania tekstu musimy skupić się tylko na implementacji części dekodera. Kluczowe komponenty, które omówimy to:

- Tokenizacja — dzielenie tekstu na tokeny
- Osadzenia wejściowe — przekształcanie tokenów w wektory
- Kodowanie pozycyjne — zachowywanie informacji o kolejności sekwencji
- Maskowanie — zapobieganie modelowi zaglądania do przodu lub patrzenia na tokeny wypełniające
- Multi-headed self-attention — łączenie różnych tokenów wejściowych
- Stos dekodera — udoskonalanie zrozumienia na wielu warstwach
- Language model head — przewidywanie prawdopodobieństw tokenów

Dzięki stopniowej implementacji każdego z tych elementów uzyskamy pełny model generowania tekstu. Wyjaśnię oba pojęcia i pokażę konkretny kod dla każdej części.

Celem jest zapewnienie intuicyjnego zrozumienia, jak działają transformery. Zaczniemy od tokenizacji!

Tokenizacja tekstu

Modele przetwarzania języka naturalnego, takie jak transformery, działają na danych liczbowych. Jednak nasze surowe dane często mają formę tekstu czytelnego dla człowieka. Aby zniwelować tę lukę, stosujemy proces zwany tokenizacją.

Tokenizacja polega na rozbiciu tekstu na mniejsze fragmenty znane jako tokeny. Granularność tych tokenów może się znacznie różnić: token może być tak mały jak pojedynczy znak, tak duży jak słowo lub gdzieś pomiędzy. Wybór rozmiaru tokena zależy od konkretnego zadania i używanej metody.

Na przykład GPT-3 wykorzystuje formę tokenizacji zwaną kodowaniem par bajtów (BPE), która tokenizuje tekst na podsłowa. Te podsłowa to sekwencje znaków, które często pojawiają się razem. To podejście stanowi rozwiązanie pośrednie między tokenizacją na poziomie znaków i słów, równoważąc wydajność i wszechstronność.

Rola słownika (Vocabulary)

Tokenizacja polega na konwersji każdego tokena w tekście na unikalny identyfikator całkowity lub indeks przy użyciu słownika zwanego Vocabulary. Słownik składa się z listy wszystkich unikalnych tokenów, które model powinien rozpoznawać.

Rozważmy zdanie „I am a cat”. Jeśli nasz słownik to [“<PAD>”, “I”, “am”, “a”, “cat”, “dog”, “mouse”], każde słowo w zdaniu jest mapowane na indeks w słowniku. „I” odpowiada 1, „am” 2, „a” 3, a „cat” 4. Tak więc nasze zdanie staje się [1, 2, 3, 4] w formie liczbowej.

Radzenie sobie z danymi wejściowymi o zmiennej długości: Wypełnienie

Transformery zazwyczaj oczekują danych wejściowych o stałym rozmiarze. Jednak rzeczywiste dane tekstowe występują w sekwencjach o zmiennej długości. Aby to uwzględnić, stosujemy technikę zwaną wypełnieniem.

Wypełnienie polega na uzupełnianiu krótszych sekwencji dodatkowymi tokenami (zwykle oznaczonymi indeksem 0), aby dopasować długość najdłuższej sekwencji w partii. Na przykład, jeśli chcemy, aby wszystkie dane wejściowe miały długość sześciu tokenów, nasze przykładowe zdanie „I am a cat.” staje się [0, 0, 1, 2, 3, 4].

Rozmiar słownika zależy od złożoności języka i granularności tokenizacji. Użyjemy prostego tokenizatora, który rozpoznaje zaledwie 39 tokenów.

Z drugiej strony tokenizator GPT-3 używa słownika 50 257 tokenów. Ten obszerny słownik obejmuje szeroki zakres angielskich słów, powszechnych części słów, a nawet całe frazy, umożliwiając GPT-3 zrozumienie i generowanie bardzo zróżnicowanego i niuansowego tekstu.

Prosty przykład tokenizatora

Tworzymy bardzo prosty tokenizator, który może kodować znaki a-z, cyfry 0-9 kropkę i znak odstępu. Token wypełniający jest mapowany na 0.

Nasz słownik jest bardzo prosty i zawiera tylko 39 tokenów.

```
class Tokenizer:

    def __init__(self):
        self.dictionary = {}
        self.reverse_dictionary = {}

        # Add the padding token
        self.__add_to_dict('<pad>')

        # Add characters and numbers to the dictionary
        for i in range(10):
            self.__add_to_dict(str(i))
        for i in range(26):
            self.__add_to_dict(chr(ord('a') + i))

        # Add space and punctuation to the dictionary
        self.__add_to_dict('.')
        self.__add_to_dict(' ')

    def __add_to_dict(self, character):
        if character not in self.dictionary:
            self.dictionary[character] = len(self.dictionary)
            self.reverse_dictionary[self.dictionary[character]] = character

    def tokenize(self, text):
        return [self.dictionary[c] for c in text]

    def character_to_token(self, character):
        return self.dictionary[character]

    def token_to_character(self, token):
        return self.reverse_dictionary[token]

    def size(self):
        return len(self.dictionary)
```

Zestaw danych

Po zdefiniowaniu modelu i procesu tokenizacji następnym krokiem jest przygotowanie odpowiedniego zestawu danych do trenowania modelu. Ponieważ dążymy do prostoty, aby zilustrować proces, skonstruujmy zestaw danych równoważny „hello world” do generowania tekstu.

Nasz zestaw danych będzie składał się z prostych, samodzielnych zdań. Każde zdanie przekazuje fakt dotyczący konkretnego zwierzęcia, taki jak „cats rule the world” lub „penguins live in the Arctic”.

```
# Create the training data
training_data = ' '.join([
    'cats rule the world',
    'dogs are the best',
    'elephants have long trunks',
    'monkeys like bananas',
    'pandas eat bamboo',
    'tigers are dangerous',
    'zebras have stripes',
    'lions are the kings of the savannah',
    'giraffes have long necks',
    'hippos are big and scary',
    'rhinos have horns',
    'penguins live in the arctic',
    'polar bears are white'
])
```

Mając przygotowane surowe dane, możemy je następnie tokenizować. Ten proces konwertuje każdy znak w naszych zdaniach treningowych na odpowiadający mu indeks tokena ze słownika

```
# Tokenize the training data
tokenized_training_data = tokenizer.tokenize(training_data)
```

Pamiętaj, że nasz model oczekuje sekwencji wejściowych o stałej długości, a nasze zdania różnią się długością. Aby to rozwiązać, użyjemy wypełnienia, a konkretnie lewego wypełnienia.

Lewostronne wypełnienie polega na dodawaniu tokenów wypełnienia („<pad>”) na początku krótszych sekwencji, dopóki nie będą one odpowiadały długości najdłuższej sekwencji w naszym zestawie danych. Wypełniając nasze sekwencje w ten sposób, zapewniamy, że każdy token w każdej sekwencji, niezależnie od jej pierwotnej długości, zostanie użyty w procesie szkolenia.

```
# Left-pad the tokenized training data
for _ in range(max_sequence_length):
    # Prepend padding tokens
    tokenized_training_data.insert(0, tokenizer.character_to_token('<pad>'))
```

Osadzanie danych wejściowych

W przetwarzaniu danych tekstowych jednym z najważniejszych kroków jest konwersja słów lub tokenów na reprezentacje numeryczne, które modele obliczeniowe mogą zrozumieć i z którymi mogą pracować. Ta transformacja jest wykonywana przez warstwę osadzania.

Wyobraź sobie tokeny jako atomowe jednostki znaczeniowe w języku — mogą to być słowa, części słów, a nawet całe frazy. Podczas tokenizacji zastępujemy te jednostki odpowiednimi liczbami całkowitymi. Jednak aby skutecznie uchwycić niuanse języka i relacje między różnymi tokenami, musimy pójść o krok dalej.

W tym miejscu pojawia się warstwa osadzania, działająca jako pomost między dyskretnymi tokenami a ciągłą przestrzenią wektorową. Przekształca każdy token, reprezentowany jako liczba całkowita, w ciągły wektor w przestrzeni wielowymiarowej.

Osadzenia i relacje semantyczne

Rozważmy uproszczony przykład. Jeśli nasze tokeny są słowami, warstwa osadzania pomaga uchwycić podobieństwo między „kotami” i „psami” poprzez umieszczenie ich odpowiadających sobie wektorów blisko siebie w przestrzeni wektorowej. Ta bliskość wynika z faktu, że „koty” i „psy” często pojawiają się w podobnych kontekstach.

Magia jednak na tym się nie kończy. Gdy układamy wiele warstw w naszym modelu, każda kolejna warstwa może uchwycić coraz bardziej złożone koncepcje. Pierwsza warstwa może rozumieć podobne zwierzęta, takie jak koty i psy. Wyższe warstwy mogą rozpoznawać role, jakie te zwierzęta odgrywają w zdaniach, takie jak bycie zwierzętami domowymi. Jeszcze wyższe warstwy mogą rozumieć bardziej złożone struktury semantyczne, takie jak koty i psy, reprezentujące różne aspekty wspólnego pojęcia (zwierzęta domowe).

Tworzenie warstwy osadzania za pomocą PyTorch

W PyTorch klasa `nn.Embedding` pomaga nam utworzyć warstwę osadzania. Podczas treningu wagi tej warstwy (reprezentujące nasze osadzenia tokenów) są aktualizowane i dostrajane, aby lepiej uchwycić relacje semantyczne między różnymi tokenami.

Oto uproszczony fragment kodu ilustrujący implementację warstwy osadzania tokenów:

```
class TokenEmbedding(torch.nn.Module):
    """
    PyTorch module that converts tokens into embeddings.

    Input dimension is: (batch_size, sequence_length)
    Output dimension is: (batch_size, sequence_length, d_model)
    """

    def __init__(self, d_model, number_of_tokens):
        super().__init__()
        self.embedding_layer = torch.nn.Embedding(
            num_embeddings=number_of_tokens,
            embedding_dim=d_model
        )

    def forward(self, x):
        return self.embedding_layer(x)
```

W powyższym kodzie:

- `number_of_tokens` wskazuje całkowitą liczbę unikalnych tokenów, jakie nasz model może napotkać na wejściu. Liczba ta zwykle jest równa rozmiarowi naszego słownika tokenów.
- `d_model` określa rozmiar (wymiarowość) wektorów osadzania. Większe wymiary pozwalają nam zakodować więcej informacji o każdym tokenie, ale także zwiększają złożoność obliczeniową i czas wymagany do trenowania modelu.

Dane wejściowe do naszego modułu `TokenEmbedding` to partia sekwencji, przy czym każdy token jest reprezentowany przez liczbę całkowitą. Natomiast dane wyjściowe to partia tych samych sekwencji, ale każda liczba całkowita jest teraz zastąpiona bogatym, wielowymiarowym wektorem zawierającym informacje semantyczne.

Kodowanie pozycyjne (Positional encoding)

Dlaczego kodowanie pozycyjne jest konieczne?

Transformery działają w oparciu o ideę `self-attention`, która oblicza wyniki istotności dla elementów w sekwencji wejściowej względem siebie. Jednak transformery z natury nie uwzględniają kolejności sekwencji, ponieważ mechanizm `self-attention` traktuje elementy wejściowe niezależnie.

W zadaniach takich jak rozumienie języka naturalnego kolejność elementów ma kluczowe znaczenie. Na przykład zdania „The cat chases the mouse” i „The mouse chases the cat” mają zupełnie inne znaczenia. Bez uwzględnienia informacji o pozycji transformer może zinterpretować te zdania jako równoważne. Dlatego potrzebujemy kodowania pozycyjnego – aby dołączyć kolejności sekwencji w transformerach.

Jak działa kodowanie pozycyjne?

W oryginalnym modelu transformera kodowanie pozycyjne jest wykonywane poprzez utworzenie wektora dla każdej pozycji w sekwencji i dodanie go do odpowiadającego mu wektora wejściowego. Robi się to, aby model mógł nauczyć się wykorzystywać te wektory pozycji do zrozumienia kolejności elementów w sekwencji.

Dokładniej rzecz biorąc, kodowanie pozycyjne dla pozycji „*p*” i wymiaru „*i*” w sekwencji wejściowej oblicza się przy użyciu następujących funkcji sinus i cosinus:

$$PE(p, 2i) = \sin(p / 10000^{(2i/d_model)})$$

$$PE(p, 2i + 1) = \cos(p / 10000^{(2i/d_model)})$$

Tutaj „`d_model`” to wymiarowość wektorów wejściowych i wyjściowych w modelu. Większy „`d_model`” oznaczałby, że każde słowo jest reprezentowane przez większy wektor, co pozwala na bardziej złożone reprezentacje, ale wymaga również większych zasobów obliczeniowych. Funkcje `sin` i `cos` są stosowane naprzemiennie do każdego wymiaru wektora kodowania pozycyjnego, tworząc wzór, którego model może się nauczyć i rozpoznać.

Wynikowe kodowania pozycyjne wahają się od -1 do 1 i mają długość fali, która zwiększa się z każdym wymiarem. Ten wzór pozwala modelowi rozróżniać różne pozycje i generalizować do długości sekwencji nie napotkanych podczas treningu.

Te kodowania pozycyjne są dodawane do osadzeń wejściowych przed przetworzeniem przez transformator. Pozwala to transformatorowi nauczyć się wykorzystywać te informacje pozycyjne w sposób, który najlepiej odpowiada zadaniu, w którym jest trenowany. Na przykład model może nauczyć się zwracać większą uwagę na sąsiednie słowa podczas interpretacji zdań, uchwytując pojęcie, że w wielu językach sąsiednie słowa często mają większe znaczenie składniowe i semantyczne względem siebie.

Kod do utworzenia modułu, który tworzy i dodaje takie osadzanie pozycyjne, można zapisać w następujący sposób:

```
class PositionalEncoding(torch.nn.Module):
    """
    Pytorch module that creates a positional encoding matrix. This matrix will later be added to the
    transformer's input embeddings to provide a sense of position of the sequence elements.
    """

    def __init__(self, d_model, max_sequence_length):
        super().__init__()
        self.d_model = d_model
        self.max_sequence_length = max_sequence_length
        self.positional_encoding = self.create_positional_encoding()

    def create_positional_encoding(self):
        """
        Creates a positional encoding matrix of size (max_sequence_length, d_model).
        """

        # Initialize positional encoding matrix
        positional_encoding = np.zeros((self.max_sequence_length, self.d_model))

        # Calculate positional encoding for each position and each dimension
        for pos in range(self.max_sequence_length):
            for i in range(0, self.d_model, 2):
                # Apply sin to even indices in the array; indices in Python start at 0 so i is even.
                positional_encoding[pos, i] = np.sin(pos / (10000 ** ((2 * i) / self.d_model)))

                if i + 1 < self.d_model:
                    # Apply cos to odd indices in the array; we add 1 to i because indices in Python
                    positional_encoding[pos, i + 1] = np.cos(pos / (10000 ** ((2 * i) / self.d_model)))

        # Convert numpy array to PyTorch tensor and return it
        return torch.from_numpy(positional_encoding).float()

    def forward(self, x):
        """
        Adds the positional encoding to the input embeddings at the corresponding positions.
        """

        # Add positional encodings to input embeddings. The ":" indexing ensures we only add position
        # to the length of the sequence in the batch. x.size(0) is the batch size, so this is a way to
        # we're not adding extra positional encodings.
        return x + self.positional_encoding[:x.size(1), :]
```

Maskowanie

Pamiętasz tokeny wypełnienia, o których mówiliśmy wcześniej? Te tokeny wypełnienia nie niosą żadnych informacji i ważne jest, aby model nie zwracał na nie uwagi podczas przetwarzania danych wejściowych.

W tym miejscu pojawia się maskowanie. Maska jest zazwyczaj tablicą o tej samej długości co zdanie, z jedynkami na pozycjach odpowiadających rzeczywistym słowom i zerami na pozycjach odpowiadających tokenom wypełnienia.

Podczas obliczania attention scores nie chcemy uwzględniać efektu tokenów wypełnienia. Aby tego uniknąć, możemy zastosować maskę do macierzy attention scores, skutecznie ustawiając wyniki w pozycjach wypełnienia na bardzo dużą liczbę ujemną (np. $-1e9$). Powodem użycia tak dużej liczby ujemnej jest to, że wyniki te są przepuszczane przez funkcję softmax(), która przekształci te duże liczby ujemne w zera. Oznacza to, że pozycje wypełnienia nie będą miały żadnego wpływu na końcowy wynik warstwy uwagi.

Oto fragment części aplikacji maski z klasy MaskedSelfAttention:

```
# Apply the mask to the attention weights, by setting the masked tokens to a very low value.
# This will make the softmax output 0 for these values.
mask = mask.reshape(attention_weights.shape[0], 1, attention_weights.shape[2])
attention_weights = attention_weights.masked_fill(mask == 0, -1e9)
```

Ten wiersz zasadniczo stosuje maskę do tensora attention_weights. Operacja masked_fill(mask == 0, $-1e9$) mówi „dla każdej pozycji w attention_weights, której odpowiadająca pozycja w masce wynosi zero (tj. token wypełniający), zamień wartość attention_weights na $-1e9$ ”. Gwarantuje to, że tokeny wypełniające nie otrzymają żadnej uwagi.

W idealnym przypadku (nie zawsze), podczas przewidywania każdego nowego słowa, dekodery powinien mieć dostęp tylko do słów, które pojawiły się przed nim w sekwencji, a nie do tych, które pojawiają się po nim. Dlaczego? Ponieważ w rzeczywistych zastosowaniach, takich jak tłumaczenie zdania, nie mamy informacji o przyszłości. Proces jest wykonywany słowo po słowie od początku do końca. Jest to znane jako właściwość autoregresyjna: przewidywanie w pewnym kroku czasowym zależy tylko od danych wejściowych i wyjściowych w poprzednich krokach czasowych.

Aby wymusić tę właściwość podczas trenowania modelu, używamy mechanizmu zwanego kolejną maską. Ta maska zapewnia, że dla dowolnego danego słowa model nie może zwrócić uwagi (lub „zajrzeć”) na słowa, które następują po nim.

Użyjmy rzeczywistego zdania, aby to zwizualizować: „The cat sat on the mat”.

Jeśli model próbuje przewidzieć słowo na czwartej pozycji („on”), powinien brać pod uwagę tylko „The”, „cat” i „sat”. Nie powinien mieć dostępu do „the” ani „mat”. Kolejna maska pomaga nam to osiągnąć, maskując (lub ukrywając) przyszłe słowa, podczas gdy model dokonuje swojej prognozy.

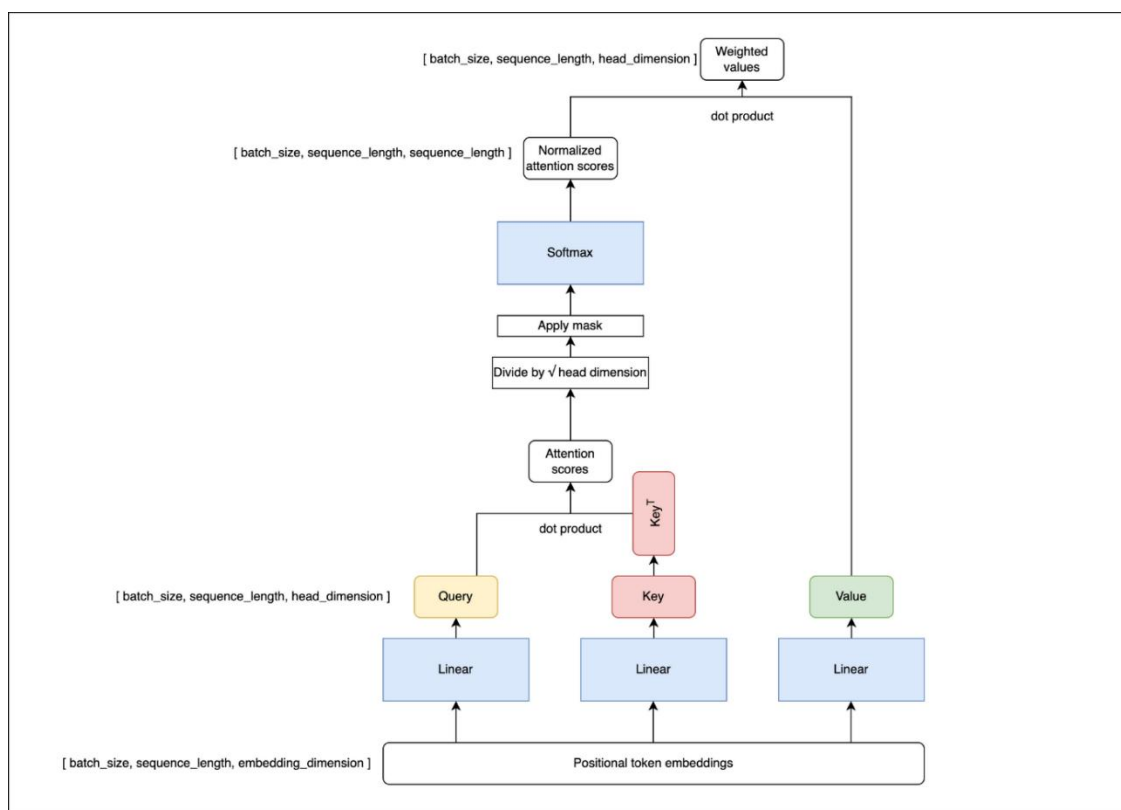
Attention

Mechanizmy uwagi (attention mechanisms) odgrywają kluczową rolę w architekturze transformera. Są one skutecznym sposobem na wyróżnienie ważnych części sekwencji wejściowej, podobnie jak my, ludzie, zwracamy większą uwagę na określone części zdania lub obrazu w zależności od kontekstu.

Wyobraźmy sobie zdanie: „The man ate a sandwich he prepared in the morning. It was topped with cheese”. Gdybyśmy chcieli zrozumieć, do czego odnosi się „It”, naturalnie zwrócilibyśmy większą uwagę na „kanapkę”, co spowodowałoby, że otrzymałaby ona wyższy wynik uwagi. To jest w zasadzie to, co robi mechanizm uwagi, obliczając wynik uwagi dla każdego słowa - bieżę pod uwagę wszystkie inne słowa w zdaniu.

Attention scores są uzyskiwane przy użyciu wektorów zapytania, klucza i wartości (query, key, and value vectors). Wektory te są tworzone przez pomnożenie osadzeń wejściowych przez wyuczone macierze (jest to w zasadzie transformacja liniowa). Wektory te pomagają obliczyć attention scores, określając wpływ jednego tokena na inny.

Następnie wektory zapytania, klucza i wartości są używane do obliczenia attention scores, jak na poniższym diagramie



Attention scores dla tokenów są obliczane jako iloczyn skalarny zapytania i wektorów klucza. Następnie wyniki te są zmniejszane przez podzielenie przez pierwiastek kwadratowy wymiaru zapytania/klucza/wartości (sztuczka z oryginalnego artykułu Transformer dla bardziej stabilnych gradientów).

Ważne jest, aby zauważyć, że uwagę można zamaskować. W przypadku zamaskowanych pozycji wynik uwagi zostanie ustawiony na -infinity, co sprawi, że pozycje te praktycznie nie będą istnieć w kolejnych obliczeniach.

Na koniec wyniki uwagi są normalizowane za pomocą funkcji softmax, aby znalazły się między 0 a 1 i sumowały się do 1. Znormalizowane wyniki uwagi są następnie mnożone przez wektory wartości i sumowane w celu uzyskania wyniku warstwy uwagi.

W PyTorch podstawowa warstwa uwagi wyglądałaby tak:

```

class MaskedSelfAttention(torch.nn.Module):
    """
    Pytorch module for a self attention layer.
    This layer is used in the MultiHeadedSelfAttention module.

    Input dimension is: (batch_size, sequence_length, embedding_dimension)
    Output dimension is: (batch_size, sequence_length, head_dimension)
    """

    def __init__(self, embedding_dimension, head_dimension):
        super().__init__()
        self.embedding_dimension = embedding_dimension
        self.head_dimension = head_dimension
        self.query_layer = torch.nn.Linear(embedding_dimension, self.head_dimension)
        self.key_layer = torch.nn.Linear(embedding_dimension, self.head_dimension)
        self.value_layer = torch.nn.Linear(embedding_dimension, self.head_dimension)
        self.softmax = torch.nn.Softmax(dim=-1)

    def forward(self, x, mask):
        """
        Compute the self attention.

        x dimension is: (batch_size, sequence_length, embedding_dimension)
        output dimension is: (batch_size, sequence_length, head_dimension)
        mask dimension is: (batch_size, sequence_length)

        mask values are: 0 or 1. 0 means the token is masked, 1 means the token is not masked.
        """

        # x dimensions are: (batch_size, sequence_length, embedding_dimension)
        # query, key, value dimensions are: (batch_size, sequence_length, head_dimension)
        query = self.query_layer(x)
        key = self.key_layer(x)
        value = self.value_layer(x)

        # Calculate the attention weights.
        # attention_weights dimensions are: (batch_size, sequence_length, sequence_length)
        attention_weights = torch.matmul(query, key.transpose(-2, -1))

        # Scale the attention weights.
        attention_weights = attention_weights / np.sqrt(self.head_dimension)

        # Apply the mask to the attention weights, by setting the masked tokens to a very low value.
        # This will make the softmax output 0 for these values.
        mask = mask.reshape(attention_weights.shape[0], 1, attention_weights.shape[2])
        attention_weights = attention_weights.masked_fill(mask == 0, -1e9)

        # Softmax makes sure all scores are between 0 and 1 and the sum of scores is 1.
        # attention_scores dimensions are: (batch_size, sequence_length, sequence_length)
        attention_scores = self.softmax(attention_weights)

        # The attention scores are multiplied by the value
        # Values of tokens with high attention score get highlighted because they are multiplied by a large value
        # and tokens with low attention score get drowned out because they are multiplied by a small value
        # Output dimensions are: (batch_size, sequence_length, head_dimension)
        return torch.bmm(attention_scores, value)

```

Wielogłowicowa uwaga dla różnych perspektyw

Mechanizm uwagi, jak już omawialiśmy, pozwala modelowi skupić się na różnych częściach sekwencji wejściowej podczas generowania każdego słowa w sekwencji wyjściowej. Ale możemy wprowadzić

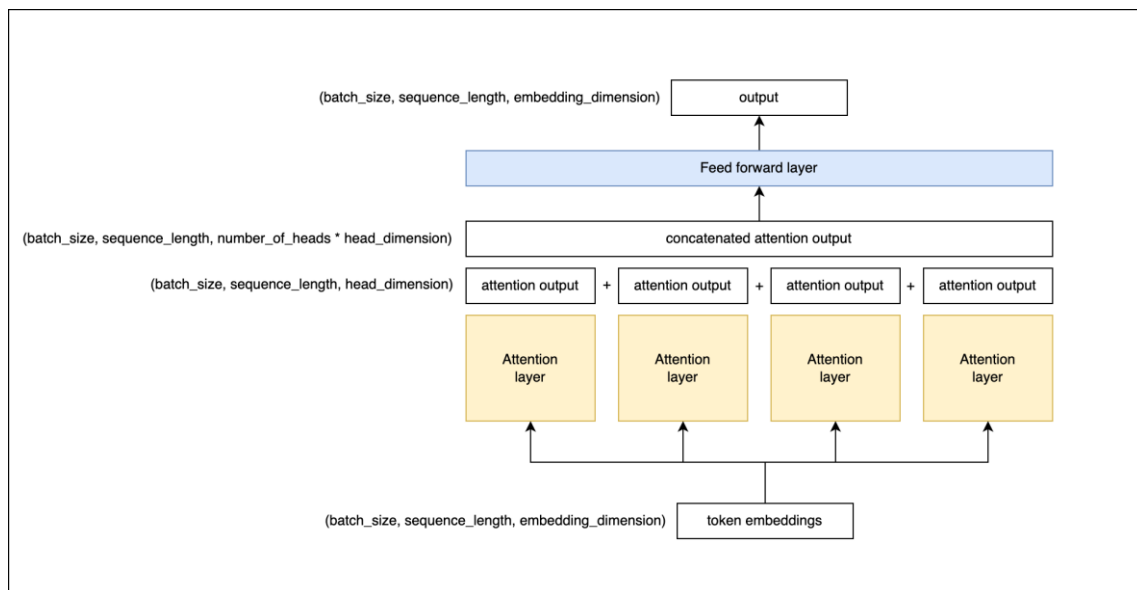
dotkliwe ulepszenie do tego systemu, aby uczynić go jeszcze bardziej wydajnym. Zamiast mieć jedną „perspektywę uwagi”, dlaczego nie mieć wielu? To jest idea stojąca za wielogłową uwagą.

Wyobraź sobie klasę uczniów, którzy wszyscy patrzą na ten sam obraz, ale każdy uczeń zauważa i skupia się na innych szczegółach — jeden może zauważyć kolory, inny kształty, inny ogólną kompozycję. Każdy z nich wnosi swoją własną, unikalną „perspektywę” do zrozumienia obrazu jako całości.

Model Transformer robi coś podobnego z wielogłową uwagą. Ma nie jeden, ale wiele zestawów (lub „główek”) mechanizmu uwagi, z których każdy niezależnie koncentruje się na różnych częściach danych wejściowych. Każda głowa może nauczyć się zwracać uwagę na różne pozycje sekwencji wejściowej i wyodrębniać różne typy informacji. Na przykład jeden mózg może nauczyć się koncentrować na aspektach składniowych języka (takich jak gramatyka), podczas gdy inny może uczyć się informacji semantycznych (takich jak znaczenie).

Oto jak to działa:

Każda głowica oblicza własne macierze zapytań, kluczy i wartości z danych wejściowych, tak jak w mechanizmie uwagi pojedynczej głowicy. Następnie obliczają swoje wyniki uwagi i generują dane wyjściowe. Jednak zamiast być przetwarzane indywidualnie, dane wyjściowe każdej głowicy są łączone ze sobą, a następnie przekształcane za pomocą warstwy liniowej w celu wygenerowania danych wyjściowych.



Multi-head self attention

Oto kod klasy MaskedMultiHeadedSelfAttention:

```

class MaskedMultiHeadedSelfAttention(torch.nn.Module):
    """
    Pytorch module for a multi head attention layer.

    Input dimension is: (batch_size, sequence_length, embedding_dimension)
    Output dimension is: (batch_size, sequence_length, embedding_dimension)
    """

    def __init__(self, embedding_dimension, number_of_heads):
        super().__init__()
        self.embedding_dimension = embedding_dimension
        self.head_dimension = embedding_dimension // number_of_heads
        self.number_of_heads = number_of_heads

        # Create the self attention modules
        self.self_attentions = torch.nn.ModuleList(
            [MaskedSelfAttention(embedding_dimension, self.head_dimension) for _ in range(number_of_heads)]

        # Create a linear layer to combine the outputs of the self attention modules
        self.output_layer = torch.nn.Linear(number_of_heads * self.head_dimension, embedding_dimension)

    def forward(self, x, mask):
        """
        Compute the multi head attention.

        x dimensions are: (batch_size, sequence_length, embedding_dimension)
        mask dimensions are: (batch_size, sequence_length)
        mask values are: 0 or 1. 0 means the token is masked, 1 means the token is not masked.
        """

        # Compute the self attention for each head
        # self_attention_outputs dimensions are:
        # (number_of_heads, batch_size, sequence_length, head_dimension)
        self_attention_outputs = [self_attention(x, mask) for self_attention in self.self_attentions]

        # Concatenate the self attention outputs
        # self_attention_outputs_concatenated dimensions are:
        # (batch_size, sequence_length, number_of_heads * head_dimension)
        concatenated_self_attention_outputs = torch.cat(self_attention_outputs, dim=2)

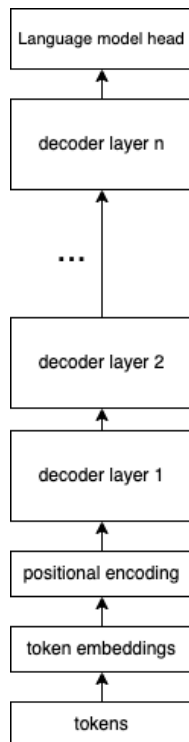
        # Apply the output layer to the concatenated self attention outputs
        # output dimensions are: (batch_size, sequence_length, embedding_dimension)
        return self.output_layer(concatenated_self_attention_outputs)

```

Warstwa dekodera

Model składa się z kilku warstw dekodera. Każdy dekodery przyjmuje dane wyjściowe poprzedniego dekodera jako dane wejściowe. Pierwszy dekodery przyjmuje warstwę kodowania pozycyjnego jako dane wejściowe. Ostatnia warstwa to nagłówek modelu języka, który będzie wyprowadzał prawdopodobieństwa następnych tokenów.

W przypadku modelu autoregresyjnego, takiego jak GPT-2, każda warstwa dekodera składa się z mechanizmu self-attention i sieci neuronowej typu feed-forward. Mechanizm self-attention pozwala modelowi ważyć znaczenie różnych tokenów w danych wejściowych podczas przewidywania następnego tokena, podczas gdy sieć typu feed-forward umożliwia modelowi naukę bardziej tradycyjnych, pozycyjnych relacji między tokenami.

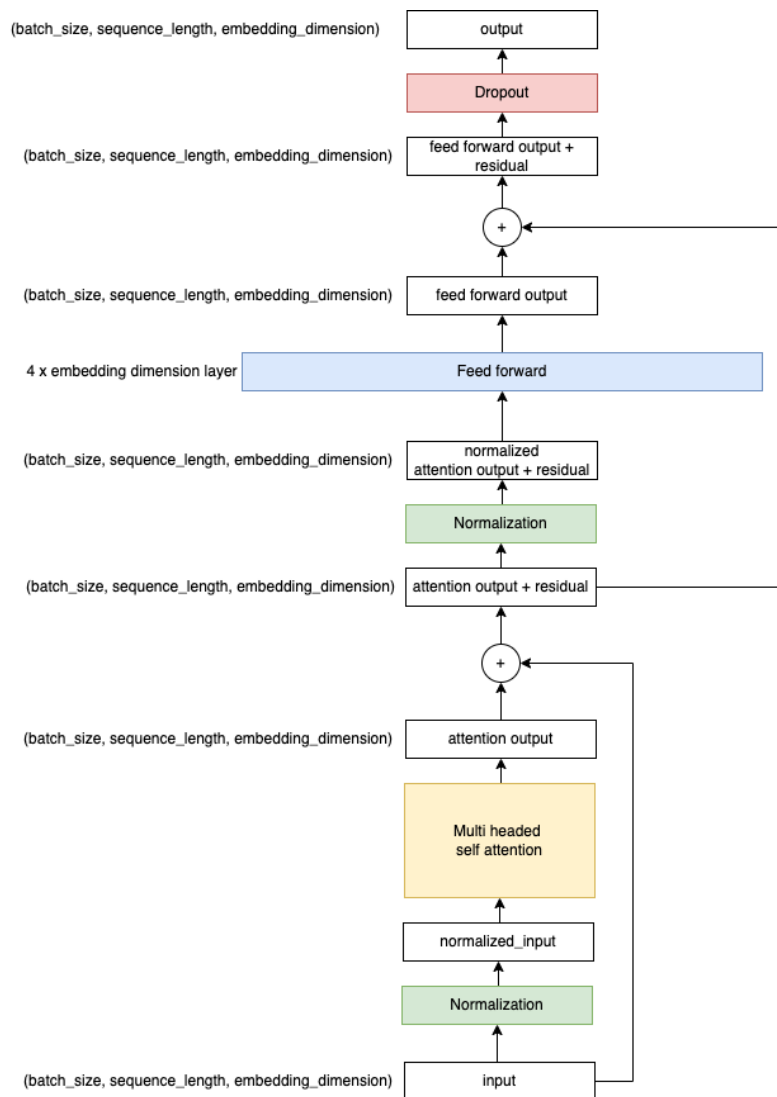


Multiple decoder layer stacked

Gdy układamy wiele z tych warstw dekodera jedna na drugiej, pozwala to modelowi udoskonalić zrozumienie danych wejściowych w kilku rundach przetwarzania. Dane wyjściowe jednej warstwy stają się danymi wejściowymi kolejnej, więc model może wykorzystać dane wyjściowe poprzednich warstw, aby poinformować o zrozumieniu bieżącej warstwy. Każda warstwa może nauczyć się reprezentować różne cechy lub aspekty danych wejściowych. Warstwy wyższego poziomu (dalej w stosie) mogą przechwytywać bardziej złożone lub abstrakcyjne cechy, które są kompozycjami prostszych cech przechwyconych przez warstwy niższego poziomu (wcześniej w stosie).

Tak więc w istocie układanie wielu warstw dekodera umożliwia modelowi przechwytywanie bardziej złożonych wzorców i zrozumienie głębszych kontekstowych relacji między danymi, co skutkuje dokładniejszymi przewidywaniami.

Więc co dokładnie robi każda warstwa dekodera?



The decoder layer

Krok 1: Normalizacja danych wejściowych i warstw

Gdy warstwa dekodera otrzymuje dane wejściowe, pierwszą rzeczą, jaką robi, jest zastosowanie warstwy normalizacji do wektorów wejściowych. Dane wejściowe do dekodera to wielowymiarowe wektory, z których każdy reprezentuje token w sekwencji.

Normalizacja warstw to kluczowy proces, który zapewnia stabilność numeryczną modelu. Zapobiega ona zbyt niemu wzrostowi tych wartości podczas treningu, łagodząc problem znany jako „wybuchające gradienty”.

Normalizacja warstw robi jednak więcej niż tylko chroni przed dużymi liczbami. Pomaga równomierniej rozłożyć dane w przestrzeni cech (Funkcje aktywacji są przecież nieliniowe), co sprawia, że proces uczenia się jest bardziej efektywny. Poprzez dostosowanie osadzeń tak, aby każdy wymiar miał średnią 0 i odchylenie standardowe 1, zapewnia, że żaden konkretny wymiar nie będzie miał nadmiernego wpływu na inne z powodu różnic w skali. To wyrównanie wpływu w wymiarach jest kluczowe dla zrównoważonego i wydajnego procesu uczenia się.

Krok 2: Self-Attention

Po znormalizowaniu danych wejściowych warstwa dekodera stosuje mechanizm self-attention. Pozwala to modelowi ocenić wagę każdego tokena w sekwencji w stosunku do innych, zapewniając porządek w kontekstowej reprezentacji każdego tokena.

Krok 3: Połączenia resztowe

Następnie dane wyjściowe z procesu self-attention są dodawane do oryginalnego, znormalizowanego wejścia w tzw. połączeniu resztowym. To połączenie to sprytny sposób na umożliwienie gradientom bezpośredniego przepływu przez sieć, eliminując inny powszechny problem głębokiego uczenia: „zanikające gradienty”. Problem „zanikających gradientów” odnosi się do gradientów, które wykładniczo zmniejszają się, w procesie wstecznej propagacji podczas obliczeń z powrotem przez warstwy podczas treningu. Ułatwia to modelowi naukę i dostosowywanie jego wcześniejszych warstw.

Krok 4: Kolejna runda normalizacji i sieć feed-forward

Po zastosowaniu połączenia resztowego dane wyjściowe są ponownie normalizowane, a następnie przepuszczane przez sieć neuronową typu feed-forward. Ta sieć jest w pełni połączoną warstwą, która stosuje nieliniowe transformacje (bardzo często ReLU) do danych, umożliwiając modelowi naukę złożonych relacji w danych.

Krok 5: Dropout (Renormalizacja)

Na koniec, aby zapobiec nadmiernemu dopasowaniu do danych treningowych, stosowana jest procedura dropout. W tym procesie losowo „wypadają” wagi (parametry sieci) tj. tymczasowo zeruje pewien procent połączeń w modelu podczas treningu (10% w poniższym kodzie). W ten sposób „zachęca się” model do nauki bardziej solidnej reprezentacji danych, która nie polega zbyt mocno na żadnym pojedynczym połączeniu.

Po przejściu przez wszystkie powyższe kroki, dane wyjściowe jednej warstwy dekodera stają się danymi wejściowymi dla następnej. To układanie w stosy wielu dekoderek pozwala modelowi nauczyć się przedstawiać dane na różnych poziomach abstrakcji, co jest kluczowe dla jego zdolności do *rozumienia* i generowania tekstu przypominającego tekst ludzki.

Cały schemat warstw dekodera można zobaczyć w kodzie: klasy `DecoderLayer` i `DecoderStack`, gdzie `DecoderLayer` hermetyzuje procesy w obrębie jednej warstwy, a `DecoderStack` obsługuje układanie w stosy wielu `DecoderLayers`.

```

class DecoderLayer(torch.nn.Module):
    """
    Pytorch module for an encoder layer.

    An encoder layer consists of a multi-headed self attention layer, a feed forward layer and dropout.

    Input dimension is: (batch_size, sequence_length, embedding_dimension)
    Output dimension is: (batch_size, sequence_length, embedding_dimension)
    """

    def __init__(
        self,
        embedding_dimension,
        number_of_heads,
        feed_forward_dimension,
        dropout_rate
    ):
        super().__init__()
        self.embedding_dimension = embedding_dimension
        self.number_of_heads = number_of_heads
        self.feed_forward_dimension = feed_forward_dimension
        self.dropout_rate = dropout_rate

        self.multi_headed_self_attention = MaskedMultiHeadedSelfAttention(embedding_dimension, number_of_heads, dropout_rate)
        self.feed_forward = FeedForward(embedding_dimension, feed_forward_dimension)
        self.dropout = torch.nn.Dropout(dropout_rate)
        self.layer_normalization_1 = torch.nn.LayerNorm(embedding_dimension)
        self.layer_normalization_2 = torch.nn.LayerNorm(embedding_dimension)

    def forward(self, x, mask):
        """
        Compute the encoder layer.

        x dimensions are: (batch_size, sequence_length, embedding_dimension)
        mask dimensions are: (batch_size, sequence_length)
        mask values are: 0 or 1. 0 means the token is masked, 1 means the token is not masked.
        """

        # Layer normalization 1
        normalized_x = self.layer_normalization_1(x)

        # Multi headed self attention
        attention_output = self.multi_headed_self_attention(normalized_x, mask)

        # Residual output
        residual_output = x + attention_output

        # Layer normalization 2
        normalized_residual_output = self.layer_normalization_2(residual_output)

        # Feed forward
        feed_forward_output = self.feed_forward(normalized_residual_output)

        # Dropout, only when training.
        if self.training:
            feed_forward_output = self.dropout(feed_forward_output)

        # Residual output
        return residual_output + feed_forward_output

```

DecoderStack to szereg warstw dekodujących ułożonych sekwencyjnie.


```

class DecoderStack(torch.nn.Module):
    """
    Pytorch module for a stack of decoders.
    """

    def __init__(
        self,
        embedding_dimension,
        number_of_layers,
        number_of_heads,
        feed_forward_dimension,
        dropout_rate,
        max_sequence_length
    ):
        super().__init__()
        self.embedding_dimension = embedding_dimension
        self.number_of_layers = number_of_layers
        self.number_of_heads = number_of_heads
        self.feed_forward_dimension = feed_forward_dimension
        self.dropout_rate = dropout_rate
        self.max_sequence_length = max_sequence_length

        # Create the encoder layers
        self.encoder_layers = torch.nn.ModuleList(
            [DecoderLayer(embedding_dimension, number_of_heads, feed_forward_dimension, dropout_rate,
                           range(number_of_layers))]

        def forward(self, x, mask):
            decoder_outputs = x
            for decoder_layer in self.encoder_layers:
                decoder_outputs = decoder_layer(decoder_outputs, mask)

            return decoder_outputs

```

Warstwa sprzężenia zwrotnego (feed forward layer)

```

class FeedForward(torch.nn.Module):
    """
    Pytorch module for a feed forward layer.

    A feed forward layer is a fully connected layer with a ReLU activation function in between.
    """

    def __init__(self, embedding_dimension, feed_forward_dimension):
        super().__init__()
        self.embedding_dimension = embedding_dimension
        self.feed_forward_dimension = feed_forward_dimension
        self.linear_1 = torch.nn.Linear(embedding_dimension, feed_forward_dimension)
        self.linear_2 = torch.nn.Linear(feed_forward_dimension, embedding_dimension)

    def forward(self, x):
        """
        Compute the feed forward layer.
        """
        return self.linear_2(torch.relu(self.linear_1(x)))

```

Model

Klasa LanguageModel łączy wszystkie warstwy, które omówiliśmy do tej pory: osadzenia tokenów, kodowanie pozycyjne, normalizację i stos dekodera. Przyjrzyjmy się każdemu komponentowi:

- **Osadzanie tokenów:** Model najpierw tworzy osadzenia tokenów. Każdy token (słowo, pod słowo lub znak) w sekwencji wejściowej jest reprezentowany przez wektor wielowymiarowy, który jest inicjowany losowo i uczony podczas treningu. Wymiar osadzania jest hiperparametrem.
- **Kodowanie pozycyjne:** Następnie model stosuje kodowanie pozycyjne do osadzeń tokenów. Pomaga to modelowi zrozumieć kolejność tokenów w sekwencji, co jest kluczową informacją w wielu zadaniach języka naturalnego. W GPT kodowanie pozycyjne jest osiągnięte przez dodanie wektora do każdego osadzenia tokena, który koduje pozycję tokena w sekwencji.
- **Normalizacja warstw:** Następnie model stosuje normalizację warstw do wyniku kodowania pozycyjnego. Proces normalizacji dostosowuje wektory tak, aby każdy wymiar miał średnią równą 0 i odchylenie standardowe równą 1, wyrównując wpływ każdego wymiaru i pomagając ustabilizować proces uczenia się.
- **Stos dekodera:** Znormalizowane dane wyjściowe są następnie przekazywane do stosu dekodatorów. Każdy blok dekodera w stosie składa się z mechanizmu self-attention i sieci neuronowej typu feed-forward, z normalizacją warstw i połączeniami resztowymi używanymi w całym procesie.

Po przejściu danych przez stos dekodera docieramy do ostatniej części modelu: nagłówka modelu języka.

```
class LanguageModel(torch.nn.Module):
    """
    Pytorch module for a language model.
    """

    def __init__(
        self,
        number_of_tokens, # The number of tokens in the vocabulary
        max_sequence_length=512, # The maximum sequence length to use for attention
        embedding_dimension=512, # The dimension of the token embeddings
        number_of_layers=6, # The number of decoder layers to use
        number_of_heads=4, # The number of attention heads to use
        feed_forward_dimension=None, # The dimension of the feed forward layer
        dropout_rate=0.1 # The dropout rate to use
    ):
        super().__init__()
        self.number_of_tokens = number_of_tokens
        self.max_sequence_length = max_sequence_length
        self.embedding_dimension = embedding_dimension
        self.number_of_layers = number_of_layers
        self.number_of_heads = number_of_heads

        if feed_forward_dimension is None:
            # GPT-2 paper uses 4 * embedding_dimension for the feed forward dimension
            self.feed_forward_dimension = embedding_dimension * 4
        else:
            self.feed_forward_dimension = feed_forward_dimension

        self.dropout_rate = dropout_rate

        # Create the token embedding layer
        self.token_embedding = TokenEmbedding(embedding_dimension, number_of_tokens)
```

```

# Create the positional encoding layer
self.positional_encoding = PositionalEncoding(embedding_dimension, max_sequence_length)

# Create the normalization layer
self.layer_normalization = torch.nn.LayerNorm(embedding_dimension)

# Create the decoder stack
self.decoder = DecoderStack(
    embedding_dimension=embedding_dimension,
    number_of_layers=number_of_layers,
    number_of_heads=number_of_heads,
    feed_forward_dimension=self.feed_forward_dimension,
    dropout_rate=dropout_rate,
    max_sequence_length=max_sequence_length
)

# Create the language model head
self.lm_head = LMHead(embedding_dimension, number_of_tokens)

def forward(self, x, mask):
    # Compute the token embeddings
    # token_embeddings dimensions are: (batch_size, sequence_length, embedding_dimension)
    token_embeddings = self.token_embedding(x)

    # Compute the positional encoding
    # positional_encoding dimensions are: (batch_size, sequence_length, embedding_dimension)
    positional_encoding = self.positional_encoding(token_embeddings)

    # Post embedding layer normalization
    positional_encoding_normalized = self.layer_normalization(positional_encoding)

    decoder_outputs = self.decoder(positional_encoding_normalized, mask)
    lm_head_outputs = self.lm_head(decoder_outputs)

    return lm_head_outputs

```

Language Model Head

Klasa LanguageModel kończy się **warstwą LMHead**. Ta warstwa jest zasadniczo liniową transformacją, która mapuje wielowymiarowe wyjście stosu dekodera z powrotem do wymiaru słownika tokenów.

Mówiąc prościej, wyobraź sobie, że masz słownik 10 000 unikalnych słów, a wymiar wyjściowy Twojego dekodera wynosi 512. Każde słowo w Twoim słowniku będzie miało unikalną 512-wymiarową reprezentację wektorową. Chcemy jednak przypisać prawdopodobieństwo każdemu słowu w słowniku, biorąc pod uwagę poprzedni kontekst. Tutaj wkracza LMHead.

LMHead mapuje 512-wymiarowy wektor z powrotem do 10 000-wymiarowej przestrzeni, zasadniczo przypisując wynik każdemu z 10 000 słów. Te wyniki są następnie przepuszczane przez funkcję softmax w celu przekształcenia ich w prawdopodobieństwa. Dlatego LMHead jest kluczowy w transformacji wielowymiarowego wyjścia stosu dekodera na prawdopodobieństwo, że każde słowo jest kolejnym słowem w sekwencji.

LMHead jest implementowany jako podklasa `torch.nn.Module` PyTorch. Używa warstwy liniowej (podstawowy typ warstwy sieci neuronowej, który stosuje transformację liniową do wejścia) w celu mapowania wymiaru wyjściowego dekodera na liczbę tokenów w słowniku.

Rozważmy scenariusz, w którym model przetworzył frazę „The quick brown fox”. Stos dekodera wyprowadza 512-wymiarowy wektor dla ostatniego słowa „fox”.

Wyobraźmy sobie teraz, że nasz słownik składa się tylko z pięciu słów: „jumps”, „sleeps”, „eats”, „runs” i „fox”. Chcemy określić prawdopodobieństwo, że każde z tych słów będzie następnym słowem w zdaniu.

W tym miejscu wkracza LMHead. Ta liniowa warstwa mapuje 512-wymiarowy wektor na 5-wymiarowy wektor, ponieważ mamy 5 słów w naszym słowniku. Każdy wpis w tym wektorze wyjściowym odpowiada słowu w słowniku i można go interpretować jako surowy wynik prawdopodobieństwa, że to słowo wystąpi po frazie „The quick brown fox”.

Założmy, że prawdopodobieństwa po zastosowaniu softmax wynoszą: ["jumps": 0.6, "sleeps": 0.05, "eats": 0.2, "runs": 0.13, "fox": 0.02]. Tak więc model przewiduje, że " The quick brown fox jumps " jest najbardziej prawdopodobną kontynuacją sekwencji wejściowej.

```
class LMHead(torch.nn.Module):
    """
    Pytorch module for the language model head.
    The language model head is a linear layer that maps the embedding dimension to the vocabulary size.
    """

    def __init__(self, embedding_dimension, number_of_tokens):
        super().__init__()
        self.embedding_dimension = embedding_dimension
        self.number_of_tokens = number_of_tokens
        self.linear = torch.nn.Linear(embedding_dimension, number_of_tokens)

    def forward(self, x):
        """
        Compute the language model head.

        x dimensions are: (batch_size, sequence_length, embedding_dimension)
        output dimensions are: (batch_size, sequence_length, number_of_tokens)
        """
        # Compute the linear layer
        # linear_output dimensions are: (batch_size, sequence_length, number_of_tokens)
        linear_output = self.linear(x)

        return linear_output
```

Autoregressive Wrapper

Aby umożliwić naszemu modelowi języka generowanie tekstu po jednym tokenie na raz, opakowaliśmy go w moduł autoregresyjny. W modelu autoregresyjnym dane wyjściowe z poprzednich kroków są przekazywane jako dane wejściowe do kolejnych kroków. Tutaj właśnie pojawia się AutoregressiveWrapper.

Autoregressive Wrapper przyjmuje sekwencję tokenów jako dane wejściowe, gdzie długość sekwencji jest o jeden token większa niż maksymalna dozwolona długość sekwencji. Ten dodatkowy token jest konieczny, ponieważ w modelach autoregresyjnych model jest trenowany, aby przewidywać następny token w sekwencji na podstawie poprzednich tokenów.

Na przykład rozważ zdanie: „The cat sat on the mat”. Jeśli sekwencją wejściową jest „The cat sat on the”, docelową sekwencją wyjściową (sekwencją, którą model jest trenowany, aby przewidywać) jest „cat sat on the mat”. Dlatego sekwencja wejściowa jest przesunięta o jeden krok w lewo względem sekwencji wyjściowej.

Klasa `AutoregressiveWrapper` zawiera również metodę obliczania prawdopodobieństw następnego tokena w sekwencji. Generuje te prawdopodobieństwa, stosując funkcję `softmax()` do logitów (czyli surowych, nieznormalizowanych wyników wyjściowych modelu dla każdego tokena w słowniku) powiązanych z ostatnim tokenem w sekwencji. Parametr „temperature” służy do dostosowywania ostrości rozkładu prawdopodobieństwa. Niższe temperatury sprawiają, że wynik jest bardziej deterministyczny (tj. bardziej prawdopodobne jest wybranie najbardziej prawdopodobnego tokena), podczas gdy wyższe temperatury sprawiają, że wynik jest bardziej losowy.

```
class AutoregressiveWrapper(torch.nn.Module):
    """
    Pytorch module that wraps a GPT model and makes it autoregressive.
    """

    def __init__(self, gpt_model):
        super().__init__()
        self.model = gpt_model
        self.max_sequence_length = self.model.max_sequence_length

    def forward(self, x, mask):
        """
        Autoregressive forward pass
        """
        inp, target = x[:, :-1], x[:, 1:]
        mask = mask[:, :-1]

        output = self.model(inp, mask)
        return output, target

    def next_token_probabilities(self, x, mask, temperature=1.0):
        """
        Calculate the token probabilities for the next token in the sequence.
        """
        logits = self.model(x, mask)[:, -1]

        # Apply the temperature
        if temperature != 1.0:
            logits = logits / temperature

        # Apply the softmax
        probabilities = torch.softmax(logits, dim=-1)

        return probabilities
```

Trener

Mamy model, który może nauczyć się, jak działa język, teraz musimy go faktycznie wytrenować, aby to zrobić.

- Najpierw tworzymy tokenizator, abyśmy mogli przekonwertować nasz zbiór danych na tokeny.
- Następnie tworzymy autoregresyjny model języka. Ponieważ zbiór danych jest bardzo mały, ustawię tylko `max_sequence_length` na 20, ale w praktyce lepszą wartością byłoby 512.
- Następnie podzielę tekst treningowy na sekwencje `[max_sequence_length + 1]`. Aby upewnić się, że początkowe tokeny są również brane pod uwagę, dane zostaną uzupełnione po lewej stronie.

- Następnie będziemy trenować model przez 50 epok, z rozmiarem partii wynoszącym 8. Epoka oznacza pełne przejście przez dane treningowe. Rozmiar partii oznacza, że w każdym przejściu do przodu przez model bierzemy pod uwagę 8 sekwencji z danych treningowych jednocześnie. Im większy rozmiar partii, tym lepiej model może nauczyć się wzorców w danych, ale większy rozmiar partii prowadzi również do większego wykorzystania pamięci, a czasami również do „przeuczenia”.

```
def create_training_sequences(max_sequence_length, tokenized_training_data):
    # Create sequences of length max_sequence_length + 1
    # The last token of each sequence is the target token
    sequences = []
    for i in range(0, len(tokenized_training_data) - max_sequence_length - 1):
        sequences.append(tokenized_training_data[i: i + max_sequence_length + 1])
    return sequences

def tokenize_and_pad_training_data(max_sequence_length, tokenizer, training_data):
    # Tokenize the training data
    tokenized_training_data = tokenizer.tokenize(training_data)
    for _ in range(max_sequence_length):
        # Prepend padding tokens
        tokenized_training_data.insert(0, tokenizer.character_to_token('<pad>'))
    return tokenized_training_data

tokenizer = Tokenizer()

embedding_dimension = 256
max_sequence_length = 20
number_of_tokens = tokenizer.size()

# Create the model
model = AutoregressiveWrapper(LanguageModel(
    embedding_dimension=embedding_dimension,
    number_of_tokens=number_of_tokens,
    number_of_heads=4,
    number_of_layers=3,
    dropout_rate=0.1,
    max_sequence_length=max_sequence_length
))
```

```

# Create the training data
training_data = ' '.join([
    'cats rule the world',
    'dogs are the best',
    'elephants have long trunks',
    'monkeys like bananas',
    'pandas eat bamboo',
    'tigers are dangerous',
    'zebras have stripes',
    'lions are the kings of the savannah',
    'giraffes have long necks',
    'hippos are big and scary',
    'rhinos have horns',
    'penguins live in the arctic',
    'polar bears are white'
])

tokenized_and_padded_training_data = tokenize_and_pad_training_data(max_sequence_length, tokenizer, training_data)
sequences = create_training_sequences(max_sequence_length, tokenized_and_padded_training_data)

# Train the model
optimizer = torch.optim.Adam(model.parameters(), lr=0.0001)
trainer = Trainer(model, tokenizer, optimizer)
trainer.train(sequences, epochs=100, batch_size=8)

```

Trener jest klasą pomocniczą, która przechodzi przez epoki i tasuje dane na początku każdej epoki. Dzięki temu zapobiegnie, aby partie były takie same za każdym razem, powodując nadmierne dopasowanie modelu do tych konkretnych partii.

Podczas tworzenia partii określamy również maskę. Np. wszystkie tokeny wypełniające są maskowane, co oznacza, że nie będą brane pod uwagę w kroku uwagi.

Następnie wykonujemy przejście do przodu przez model z partią. Oznacza to, że pozwalamy modelowi tworzyć prognozy przy użyciu podanych danych. Następnie prognozy są porównywane z wartością docelową, która jest sekwencją przesuniętą o 1 krok, tak aby następny token stał się widoczny. Model wyprowadza prawdopodobieństwa, który token powinien być następny. Funkcja straty wie, jaka powinna być odpowiedź. Im dalej od celu była prognoza, tym wyższa będzie wartość straty.

Po obliczeniu wartości straty model można zaktualizować. Odbywa się to poprzez obliczenie gradientów; kierunku, w którym wagi powinny być dostosowywane, aby poprawić prognozę modelu. Następnie model jest lekko dostosowywany w kierunku gradientów i można przetworzyć nową partię.

Jeśli wszystko działa zgodnie z planem, strata powinna spadać z czasem. Zwracam stratę dla każdej epoki, więc można ją wykreślić.

```

class Trainer:

    def __init__(self, model, tokenizer: Tokenizer, optimizer=None):
        super().__init__()
        self.model = model
        if optimizer is None:
            self.optimizer = torch.optim.Adam(model.parameters(), lr=0.0001)
        else:
            self.optimizer = optimizer
        self.tokenizer = tokenizer
        self.loss_function = torch.nn.CrossEntropyLoss()

    def train(self, data: List[str], epochs, batch_size):
        loss_per_epoch = []
        for epoch in range(epochs):
            losses = []

            # Shuffle the sequences
            random.shuffle(data)

            # Create batches of sequences and their respective mask.
            batches = []
            for i in range(0, len(data), batch_size):
                sequence_tensor = torch.tensor(data[i: i + batch_size], dtype=torch.long)

                # Create the mask tensor for the batch, where 1 means the token is not a padding token
                mask_tensor = torch.ones_like(sequence_tensor)
                mask_tensor[sequence_tensor == self.tokenizer.character_to_token('<pad>')] = 0

                batches.append((sequence_tensor, mask_tensor))

            # Train the model on each batch
            for batch in batches:
                self.model.train()

                # Create the input and mask tensors
                input_tensor = torch.zeros((batch_size, self.model.max_sequence_length + 1), dtype=torch.long)
                mask_tensor = torch.zeros((batch_size, self.model.max_sequence_length + 1), dtype=torch.long)

                for i, input_entry in enumerate(batch[0]):
                    input_tensor[i] = input_entry

                for i, mask_entry in enumerate(batch[1]):
                    mask_tensor[i] = mask_entry

                # Compute the model output
                model_output, target = self.model.forward(x=input_tensor, mask=mask_tensor)

                # Compute the losses
                # The loss is computed on the model output and the target
                loss = self.loss_function(model_output.transpose(1, 2), target)

                # Backpropagate the loss.
                loss.backward()

                # Clip the gradients. This is used to prevent exploding gradients.
                torch.nn.utils.clip_grad_norm_(self.model.parameters(), 0.5)

                # Update the model parameters. This is done by taking a step in the direction of the
                self.optimizer.step()

                # Reset the gradients. This is done so that the gradients from the previous batch
                # are not used in the next step.
                self.optimizer.zero_grad()

                # Append the loss to the list of losses, so that the average loss can be computed for
                losses.append(loss.item())

```

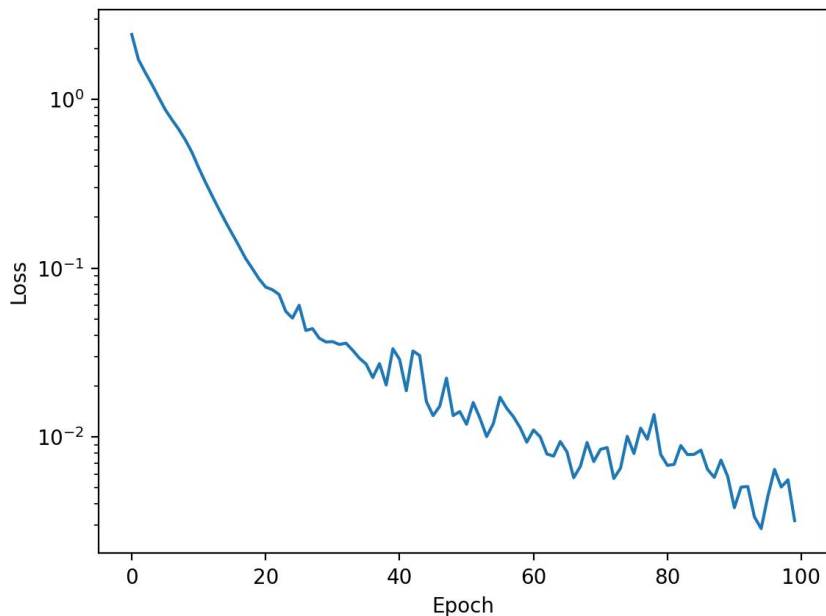


```
# Print the loss
epoch_loss = np.average(losses)
loss_per_epoch.append(epoch_loss)
print('Epoch:', epoch, 'Loss:', epoch_loss)

return loss_per_epoch
```

Ostatecznie wykres strat vs epoka powinno dać nam ładny wykres ze zmniejszającą się stratą. Jest on wykreślony w skali logarytmicznej, więc można zobaczyć mniejsze zmiany pod koniec treningu.

```
# Plot the loss per epoch in log scale
plt.plot(loss_per_epoch)
plt.yscale('log')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.show()
```



Generator

Teraz model jest wytrenowany i chcę sprawdzić, czy faktycznie nauczył się pisać. Spróbujmy wygenerować tekst na podstawie monitu „elephants”. Chcę, aby kontynuował pisanie przez 50 tokenów.

Ponieważ jedyna wzmianka o słoniach w danych treningowych to „słonie mają długie trąby”, spodziewam się, że model to napisze.

```
max_tokens_to_generate = 50
generator = Generator(model, tokenizer)
generated_text = generator.generate(
    max_tokens_to_generate=max_tokens_to_generate,
    prompt="elephants",
    padding_token=tokenizer.character_to_token('<pad>')
)
print(generated_text.replace('<pad>', ''))
```

ale najpierw musimy napisać kod dla Generatora. Pomocnicza klasa do generowania tekstu.

Najpierw przełączamy model z trybu „treningowego” na tryb „eval”. W trybie eval model nie zastosuje dropout.

Podawany przez nas monit jest konwertowany na tokeny, a następnie uzupełniany, aby miał prawidłową długość sekwencji.

Następnie będziemy autoregresyjnie generować nowe tokeny i dodawać je do sekwencji wejściowej. Po dodaniu tokena ponownie uruchamiamy nową sekwencję wejściową z dodatkowym tokenem w modelu i dołączamy nowy token. Kontynuujemy ten proces, aż zostanie osiągnięta maksymalna liczba znaków, które chcieliśmy wygenerować, lub aż wygenerujemy eos_token, czyli token końca sekwencji. Jest to token, który może zostać zdefiniowany przez użytkownika jako wskazanie, że musimy zatrzymać generowanie.

```
def pad_left(sequence, final_length, padding_token):
    return [padding_token] * (final_length - len(sequence)) + sequence

class Generator:

    def __init__(
        self,
        model,
        tokenizer):
        self.model = model
        self.tokenizer = tokenizer

    def generate(
        self,
        max_tokens_to_generate: int,
        prompt: str = None,
        temperature: float = 1.0,
        eos_token: int = None,
        padding_token: int = 0):

        self.model.eval()

        if prompt is None:
            start_tokens = [self.tokenizer.character_to_token(padding_token)]
        else:
            start_tokens = self.tokenizer.tokenize(prompt)

        input_tensor = torch.tensor(
            pad_left(
                sequence=start_tokens,
                final_length=self.model.max_sequence_length + 1,
                padding_token=padding_token
            ),
            dtype=torch.long
        )
```

```

num_dims = len(input_tensor.shape)

if num_dims == 1:
    input_tensor = input_tensor[None, :]

out = input_tensor
for _ in range(max_tokens_to_generate):

    x = out[:, -self.model.max_sequence_length:]

    mask = torch.ones_like(x)
    mask[x == padding_token] = 0

    # Compute the next token probabilities
    next_token_probabilities = self.model.next_token_probabilities(
        x=x,
        temperature=temperature,
        mask=mask
    )

    # Sample the next token from the probability distribution
    next_token = torch.multinomial(next_token_probabilities, num_samples=1)

    # Append the next token to the output
    out = torch.cat([out, next_token], dim=1)

    # If the end of sequence token is reached, stop generating tokens
    if eos_token is not None and next_token == eos_token:
        break

generated_tokens = out[0].tolist()
return ''.join([self.tokenizer.token_to_character(token) for token in generated_tokens])

```

Szkolenie zostało zakończone i generator został uruchomiony. Ufff... model faktycznie wyprowadza tekst, na którym został wytrenowany!

```

Epoch: 80 Loss: 0.0067777954955349485
Epoch: 81 Loss: 0.00686569258910962
Epoch: 82 Loss: 0.008877060552969432
Epoch: 83 Loss: 0.007847674174207406
Epoch: 84 Loss: 0.007881920627998905
Epoch: 85 Loss: 0.008343779631944086
Epoch: 86 Loss: 0.006408184001114792
Epoch: 87 Loss: 0.005755460942358089
Epoch: 88 Loss: 0.007279170352446202
Epoch: 89 Loss: 0.005830544498167001
Epoch: 90 Loss: 0.0038056843063364234
Epoch: 91 Loss: 0.00503472179825155
Epoch: 92 Loss: 0.005079620280482162
Epoch: 93 Loss: 0.0033528842711045095
Epoch: 94 Loss: 0.0028604445335356733
Epoch: 95 Loss: 0.004454014157930102
Epoch: 96 Loss: 0.006416413879308563
Epoch: 97 Loss: 0.005042898430800937
Epoch: 98 Loss: 0.005563385790036335
Epoch: 99 Loss: 0.003188125843268771
elephants have long trunks. monkeys like bananas. pandas ea

```

Oczywiście, podczas trenowania na tak małym zbiorze danych model całkowicie go przepisuje i nauczy się odtwarzać cały zbiór danych. Nie jest to to, czego zazwyczaj szukasz w modelu językowym, ale w tym teście „hello world” oznacza sukces!

Zapisywanie i ładowanie modelu

Po wytrenowaniu modelu przydatne jest, jeśli możesz go zapisać, aby nie musieć trenować nowego modelu za każdym razem.

Aby to zrobić, dodajemy następujący kod do klasy LanguageModel.

```
def save_checkpoint(self, path):
    print(f'Saving checkpoint {path}')
    torch.save({
        'number_of_tokens': self.number_of_tokens,
        'max_sequence_length': self.max_sequence_length,
        'embedding_dimension': self.embedding_dimension,
        'number_of_layers': self.number_of_layers,
        'number_of_heads': self.number_of_heads,
        'feed_forward_dimension': self.feed_forward_dimension,
        'dropout_rate': self.dropout_rate,
        'model_state_dict': self.state_dict()
    }, path)

    @staticmethod
    def load_checkpoint(path) -> 'LanguageModel':
        checkpoint = torch.load(path)
        model = LanguageModel(
            number_of_tokens=checkpoint['number_of_tokens'],
            max_sequence_length=checkpoint['max_sequence_length'],
            embedding_dimension=checkpoint['embedding_dimension'],
            number_of_layers=checkpoint['number_of_layers'],
            number_of_heads=checkpoint['number_of_heads'],
            feed_forward_dimension=checkpoint['feed_forward_dimension'],
            dropout_rate=checkpoint['dropout_rate']
        )
        model.load_state_dict(checkpoint['model_state_dict'])
        return model
```

Ponieważ używamy AutoregressiveWrapper jako klasy wygodnej, możemy wyposażyć tę klasę również w metody save i load.

```
def save_checkpoint(self, path):
    self.model.save_checkpoint(path)

    @staticmethod
    def load_checkpoint(path) -> 'AutoregressiveWrapper':
        model = LanguageModel.load_checkpoint(path)
        return AutoregressiveWrapper(model)
```

Dzięki temu możliwe jest łatwe zapisywanie i wczytywanie wytrenowanego modelu za pomocą .

```
model.save_checkpoint('./trained_model')
model = model.load_checkpoint('./trained_model')
```

Running on GPU

Jeśli dysponujesz GPU i skonfigurowano Cuda, możesz użyć GPU, aby przyspieszyć szkolenie.

Najpierw musimy zdefiniować funkcję, aby określić, czy możemy użyć GPU:

```
def get_device():
    if torch.cuda.is_available():
        return torch.device('cuda')
    else:
        return torch.device('cpu')
```

Następnie musimy przenieść model i wszystkie tensory wejściowe do naszego urządzenia. Możemy to zrobić, używając funkcji torch „to”.

```
# Model
model = AutoregressiveWrapper(LanguageModel(
    embedding_dimension=embedding_dimension,
    number_of_tokens=number_of_tokens,
    number_of_heads=4,
    number_of_layers=3,
    dropout_rate=0.1,
    max_sequence_length=max_sequence_length
)).to(get_device())

# Training input
model_output, target = self.model.forward(
    x=input_tensor.to(get_device()),
    mask=mask_tensor.to(get_device())
)

# Generation input
input_tensor = torch.tensor(
    pad_left(
        sequence=start_tokens,
        final_length=self.model.max_sequence_length + 1,
        padding_token=padding_token
    ),
    dtype=torch.long
).to(get_device())
```

Przyspieszenie uzyskane dzięki wykorzystaniu procesora graficznego może być naprawdę znaczące.